STAT3888: New Model Performance Metrics

AUTHOR
Calvin Lai

PUBLISHED
September 24, 2026

Using the new dataset, I want to try and just do cross-validation for the metrics such as log loss, AUC, and balanced accuracy for the main model, since I have already done that for the `ALT2`/`GGT2` models.

Load Data and Prepare Functions

▼ Code

```
# Initial setup
knitr::opts_chunk$set(warning = FALSE, message = FALSE,
                      fig.align='center', fig.dim=c(10, 8))

# General
library(tidyverse)
library(here)
library(knitr)
library(kableExtra)

# Modelling and CV
library(nnet)
library(caret)
library(pROC)

# Data
dat.full = readRDS(here("Objects", "dat_full.rds"))

# Helper function for metrics
metrics_summary = function(data, lev = NULL, model = NULL) {

  metrics = multiClassSummary(
    data = data,
    lev = lev,
    model = model
  )

  c(
    AUC = unname(metrics["AUC"]),
    LogLoss = unname(metrics["logLoss"]),
```

```
    BalancedAccuracy = unname(metrics["Mean_Balanced_Accuracy"])
  )
}

# Helper function for getting the final metrics
get_best_metrics = function(model) {

  best = model$results[
    model$results$decay == model$bestTune$decay,
    c("AUC", "LogLoss", "BalancedAccuracy"),
    drop = FALSE
  ]

  return(best)
}

# Create folds for reuse later
set.seed(2992)
ALT.folds = createFolds(dat.full$ALT3, k = 10, returnTrain = TRU
GGT.folds = createFolds(dat.full$GGT3, k = 10, returnTrain = TRU

# CV Settings
ALT.cv_control = trainControl(method = "cv", index = ALT.folds,
                              summaryFunction = metrics_summary,
                              classProbs = TRUE)
GGT.cv_control = trainControl(method = "cv", index = GGT.folds,
                              summaryFunction = metrics_summary,
                              classProbs = TRUE)
```

Now, we do the cross-validated fitting, and take metrics.

▼ Code

```
# Fit MLR Models
ALT.mlr.cv = train(ALT3 ~ ExerciseGuidelines + Sex + AlcoholRisk
                     SmokingDuration + NonSchoolQuals + IncomeDe
                   data = dat.full, method = "multinom",
                   trControl = ALT.cv_control, metric = "LogLoss
                   trace = FALSE)
GGT.mlr.cv = train(GGT3 ~ ExerciseGuidelines + Sex + AlcoholRisk
                     SmokingDuration + NonSchoolQuals + IncomeDe
                   data = dat.full, method = "multinom",
                   trControl = GGT.cv_control, metric = "LogLoss
                   trace = FALSE)

# Get metrics for later
ALT.mlr.metrics = get_best_metrics(ALT.mlr.cv)
```

```
GGT.mlr.metrics = get_best_metrics(GGT.mlr.cv)

mlr.metrics.table = bind_rows(
  ALT = ALT.mlr.metrics,
  GGT = GGT.mlr.metrics,
  .id = "Model"
)

kable(
  mlr.metrics.table,
  digits = 3,
  caption = "Cross-Validated Performance of Multinomial Logisti
  align = "cccc"
)
```

Cross-Validated Performance of Multinomial Logistic Regression Models

Model	AUC	LogLoss	BalancedAccuracy
ALT	0.679	0.658	0.510
GGT	0.665	0.562	0.508

So the models are not particularly much better than the `ALT2` / `GGT2` models. The balanced accuracy still hovers around the 0.5 mark and AUC and LogLoss are comparable to the logistic regression and GAM models.